Linear Regression

Nirjhor Datta

BRAC University

What is Linear Regression?

Goal: Learn a straight-line relationship between input and output.

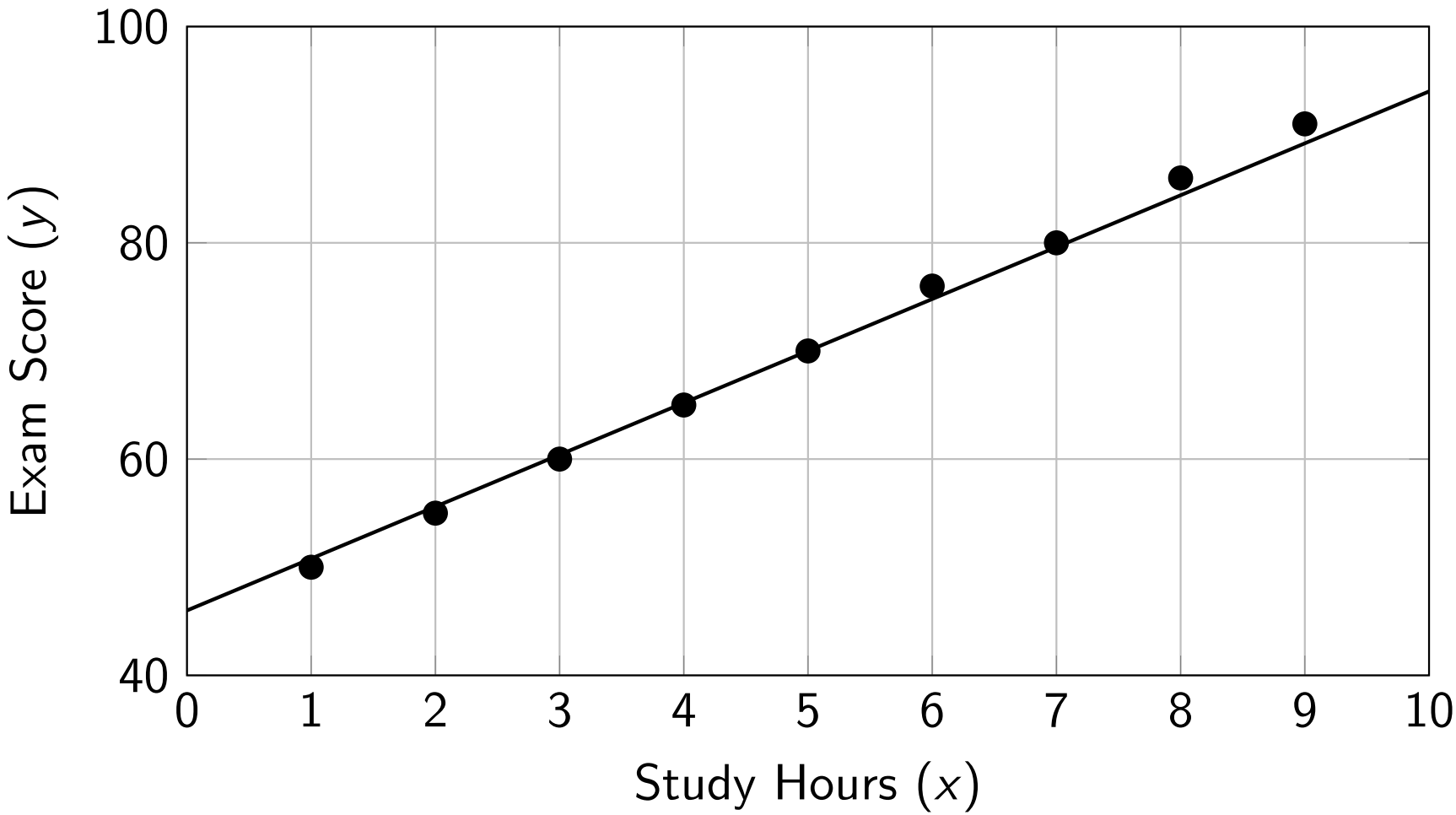
$$\hat{y} = wx + b$$

- x = input feature
- $\hat{y}$ = predicted output
- w = slope of the line
- b = intercept

Interpretation:

- As x changes, $\hat{y}$ changes linearly.
- Linear regression tries to find the **best-fitting line**.

Visual Example: Best-Fit Line



Example: More study hours generally lead to higher exam scores.

How Does the Model Make Predictions?

Suppose the learned line is:

$$\hat{y} = 4.8x + 46$$

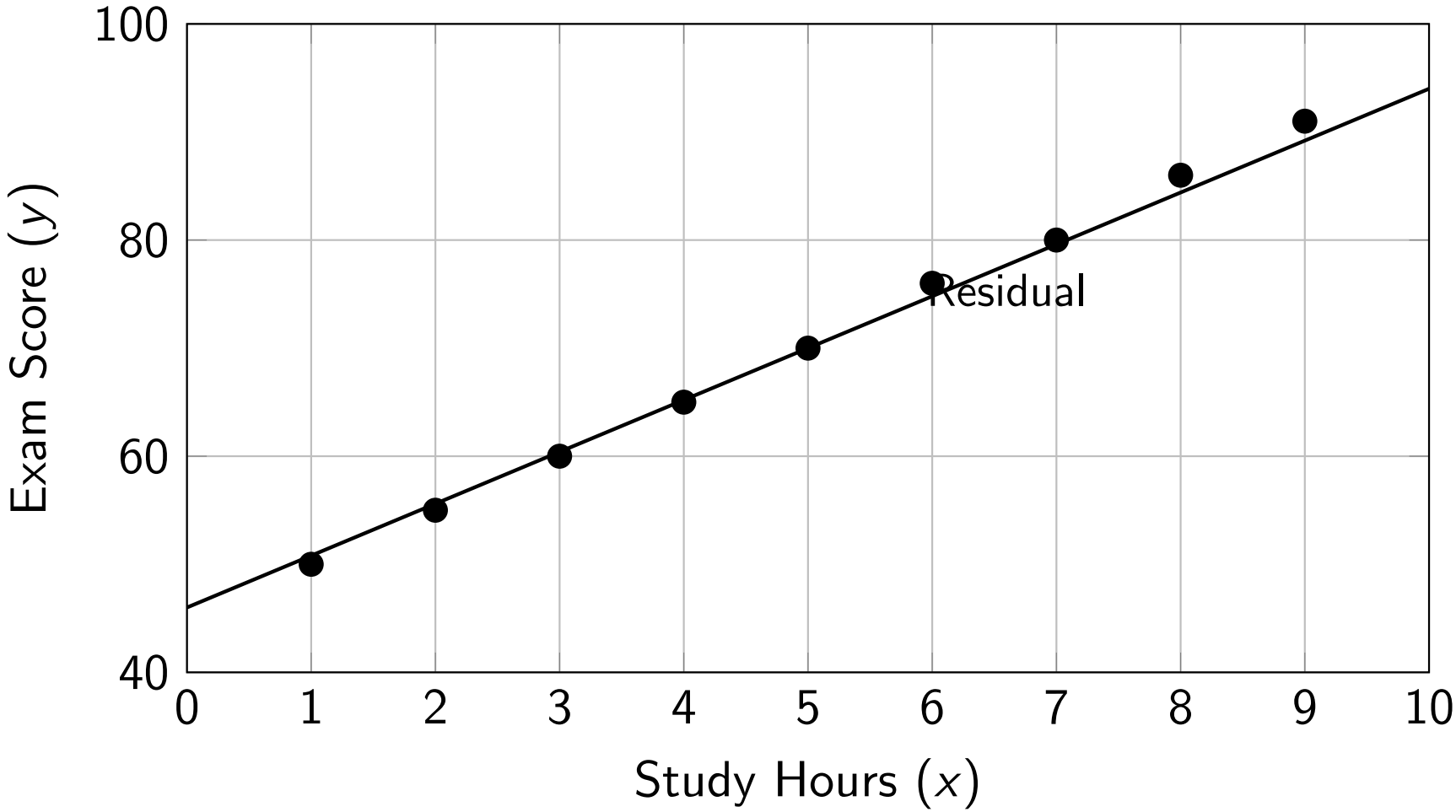
If a student studies for $x = 5$ hours, then:

$$\hat{y} = 4.8(5) + 46 = 70$$

Meaning:

- The model predicts an exam score of 70.
- The line allows us to estimate outputs for new inputs.

Residual Error: Distance from the Line



Residual:

$$\text{Residual} = y - \hat{y}$$

It measures how far a data point is from the regression line

Training Objective

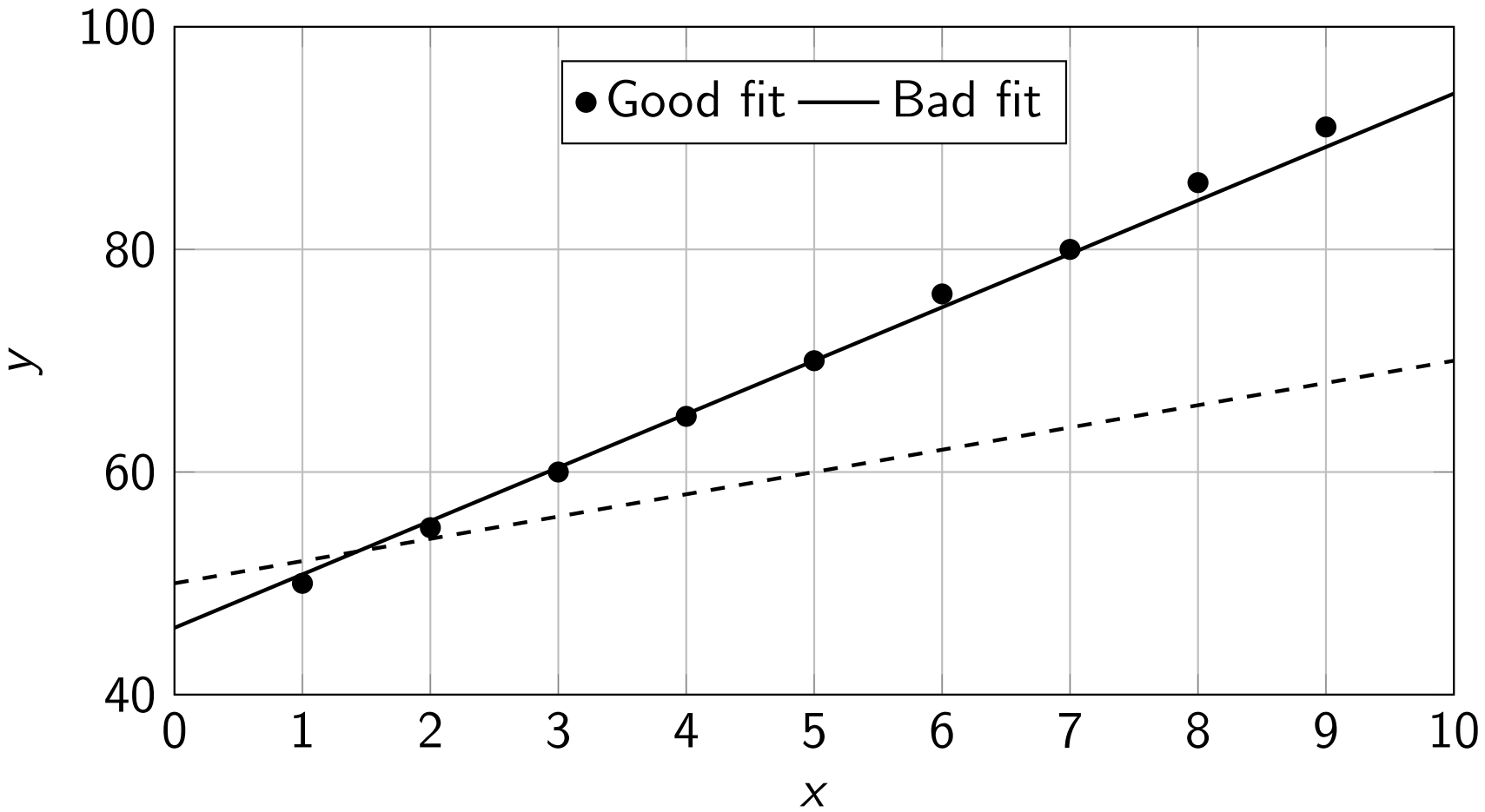
Linear regression chooses w and b such that prediction errors are as small as possible.

Common loss function: Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- y_i = true value
- $\hat{y}_i$ = predicted value
- Square is used so that large errors are penalized more

Good Fit vs Bad Fit



A good regression line stays closer to the data points overall.

When Does Linear Regression Work Well?

Works well when:

- Relationship between variables is approximately linear
- Noise is not too large
- Trend can be captured by a straight line

Examples:

- Study hours vs exam score
- Advertising budget vs sales
- House size vs price

Limitations of Linear Regression

Problem: Not all relationships are linear.

- Cannot model curves well
- Sensitive to outliers
- Assumes a simple straight-line pattern

This is why AI moves beyond linear regression to:

- Polynomial models
- Logistic regression
- Neural networks

Loss Function: Measuring Error

Goal:

How do we quantify how wrong our model is?

Loss Function: Measuring Error

Goal:

How do we quantify how wrong our model is?

Given:

$$\hat{y}_i = wx_i + b$$

Loss Function: Measuring Error

Goal:

How do we quantify how wrong our model is?

Given:

$$\hat{y}_i = wx_i + b$$

Residual (error):

$$e_i = y_i - \hat{y}_i$$

Loss Function: Measuring Error

Goal:

How do we quantify how wrong our model is?

Given:

$$\hat{y}_i = wx_i + b$$

Residual (error):

$$e_i = y_i - \hat{y}_i$$

Mean Squared Error (MSE):

$$\mathcal{L}(w, b) = \frac{1}{n} \sum_{i=1}^{n} (y_i - (wx_i + b))^2$$

Loss Function: Measuring Error

Goal:

How do we quantify how wrong our model is?

Given:

$$\hat{y}_i = wx_i + b$$

Residual (error):

$$e_i = y_i - \hat{y}_i$$

Mean Squared Error (MSE):

$$\mathcal{L}(w, b) = \frac{1}{n} \sum_{i=1}^{n} (y_i - (wx_i + b))^2$$

Why square the error?

- Penalizes large errors more
- Always positive
- Smooth and differentiable (important for optimization)

Learning = Optimization Problem

Objective:

$$(w^*, b^*) = \arg\min_{w,b} \mathcal{L}(w, b)$$

Learning = Optimization Problem

Objective:

$$(w^*, b^*) = \arg\min_{w,b} \mathcal{L}(w, b)$$

Interpretation:

- Find parameters that minimize prediction error

Learning = Optimization Problem

Objective:

$$(w^*, b^*) = \arg\min_{w,b} \mathcal{L}(w, b)$$

Interpretation:

- Find parameters that minimize prediction error

Key Insight:

Training a model = solving an optimization problem

Learning = Optimization Problem

Objective:

$$(w^*, b^*) = \arg\min_{w,b} \mathcal{L}(w, b)$$

Interpretation:

- Find parameters that minimize prediction error

Key Insight:

Training a model = solving an optimization problem

Question:

- How do we find the minimum?

Gradient Descent: How Learning Happens

Idea:

Move parameters in the direction that reduces loss

Gradient Descent: How Learning Happens

Idea:

Move parameters in the direction that reduces loss

Update rule:

$$w = w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

$$b = b - \eta \frac{\partial \mathcal{L}}{\partial b}$$

Gradient Descent: How Learning Happens

Idea:

Move parameters in the direction that reduces loss

Update rule:

$$w = w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

$$b = b - \eta \frac{\partial \mathcal{L}}{\partial b}$$

Where:

- η = learning rate (step size)
- Gradient = direction of steepest increase

Gradient Descent: How Learning Happens

Idea:

Move parameters in the direction that reduces loss

Update rule:

$$w = w - \eta \frac{\partial \mathcal{L}}{\partial w}$$

$$b = b - \eta \frac{\partial \mathcal{L}}{\partial b}$$

Where:

- $\eta =$ learning rate (step size)
- Gradient $=$ direction of steepest increase

So we move in:

$$-\nabla \mathcal{L} \quad \text{(steepest decrease)}$$

Gradient for Linear Regression

Loss:

$$\mathcal{L}(w, b) = \frac{1}{n} \sum (y_i - (wx_i + b))^2$$

Gradient for Linear Regression

Loss:

$$\mathcal{L}(w, b) = \frac{1}{n} \sum (y_i - (wx_i + b))^2$$

Gradient w.r.t w:

$$\frac{\partial \mathcal{L}}{\partial w} = -\frac{2}{n} \sum x_i (y_i - \hat{y}_i)$$

Gradient for Linear Regression

Loss:

$$\mathcal{L}(w, b) = \frac{1}{n}\sum(y_i - (wx_i + b))^2$$

Gradient w.r.t w:

$$\frac{\partial \mathcal{L}}{\partial w} = -\frac{2}{n}\sum x_i(y_i - \hat{y}_i)$$

Gradient w.r.t b:

$$\frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{n}\sum(y_i - \hat{y}_i)$$

Gradient for Linear Regression

Loss:

$$\mathcal{L}(w, b) = \frac{1}{n} \sum (y_i - (wx_i + b))^2$$

Gradient w.r.t w:

$$\frac{\partial \mathcal{L}}{\partial w} = -\frac{2}{n} \sum x_i (y_i - \hat{y}_i)$$

Gradient w.r.t b:

$$\frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{n} \sum (y_i - \hat{y}_i)$$

Insight:

- Error $\times$ input determines update

Gradient Descent: Intuition

Think of a landscape:

- Loss function = height
- Parameters (w, b) = position

Gradient Descent: Intuition

Think of a landscape:

- Loss function = height
- Parameters (w, b) = position

Goal:

Reach the lowest point (minimum loss)

Gradient Descent: Intuition

Think of a landscape:

- Loss function = height
- Parameters (w, b) = position

Goal:

Reach the lowest point (minimum loss)

Process:

- Start somewhere
- Look at slope
- Move downhill
- Repeat until convergence

Gradient Descent: Intuition

Think of a landscape:

- Loss function = height
- Parameters (w, b) = position

Goal:

Reach the lowest point (minimum loss)

Process:

- Start somewhere
- Look at slope
- Move downhill
- Repeat until convergence

Key idea:

- Gradient tells us which direction to move

From Single Input to Multiple Inputs

Model:

$$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id} + b$$

From Single Input to Multiple Inputs

Model:

$$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id} + b$$

Compact form:

$$\hat{y}_i = \mathbf{w}^T \mathbf{x}_i + b$$

where

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}, \qquad \mathbf{x}_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{id} \end{bmatrix}$$

From Single Input to Multiple Inputs

Model:

$$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_d x_{id} + b$$

Compact form:

$$\hat{y}_i = \mathbf{w}^T \mathbf{x}_i + b$$

where

$$\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_d \end{bmatrix}, \qquad \mathbf{x}_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{id} \end{bmatrix}$$

Loss function:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big)^2$$

Gradient Derivation w.r.t. One Weight

Loss:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big)^2$$

Gradient Derivation w.r.t. One Weight

Loss:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big)^2$$

To compute the gradient for one parameter w_j:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial}{\partial w_j} \left[\frac{1}{n} \sum_{i=1}^{n} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big)^2 \right]$$

Gradient Derivation w.r.t. One Weight

Loss:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big)^2$$

To compute the gradient for one parameter w_j:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial}{\partial w_j} \left[\frac{1}{n} \sum_{i=1}^{n} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big)^2 \right]$$

Move the derivative inside the summation:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{1}{n} \sum_{i=1}^{n} \frac{\partial}{\partial w_j} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big)^2$$

Gradient Derivation w.r.t. One Weight

Loss:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big)^2$$

To compute the gradient for one parameter w_j:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{\partial}{\partial w_j} \left[\frac{1}{n} \sum_{i=1}^{n} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big)^2 \right]$$

Move the derivative inside the summation:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{1}{n} \sum_{i=1}^{n} \frac{\partial}{\partial w_j} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big)^2$$

Apply the chain rule:

$$= \frac{1}{n} \sum_{i=1}^{n} 2 \left(y_i - \hat{y}_i \right) \frac{\partial}{\partial w_j} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big)$$

Finishing the Derivation

Recall:

$$\mathbf{w}^T \mathbf{x}_i = \sum_{k=1}^{d} w_k x_{ik}$$

So,

$$\frac{\partial}{\partial w_j} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big) = -x_{ij}$$

Finishing the Derivation

Recall:

$$\mathbf{w}^T \mathbf{x}_i = \sum_{k=1}^{d} w_k x_{ik}$$

So,

$$\frac{\partial}{\partial w_j} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big) = -x_{ij}$$

Substitute back:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{1}{n} \sum_{i=1}^{n} 2(y_i - \hat{y}_i)(-x_{ij})$$

Finishing the Derivation

Recall:

$$\mathbf{w}^T \mathbf{x}_i = \sum_{k=1}^{d} w_k x_{ik}$$

So,

$$\frac{\partial}{\partial w_j} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big) = -x_{ij}$$

Substitute back:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{1}{n} \sum_{i=1}^{n} 2(y_i - \hat{y}_i)(-x_{ij})$$

Therefore,

$$\boxed{\frac{\partial \mathcal{L}}{\partial w_j} = -\frac{2}{n} \sum_{i=1}^{n} x_{ij}(y_i - \hat{y}_i)}$$

Finishing the Derivation

Recall:

$$\mathbf{w}^T \mathbf{x}_i = \sum_{k=1}^{d} w_k x_{ik}$$

So,

$$\frac{\partial}{\partial w_j} \Big(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \Big) = -x_{ij}$$

Substitute back:

$$\frac{\partial \mathcal{L}}{\partial w_j} = \frac{1}{n} \sum_{i=1}^{n} 2(y_i - \hat{y}_i)(-x_{ij})$$

Therefore,

$$\boxed{\frac{\partial \mathcal{L}}{\partial w_j} = -\frac{2}{n} \sum_{i=1}^{n} x_{ij}(y_i - \hat{y}_i)}$$

Interpretation:

Gradient Derivation w.r.t. Bias

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \right)^2$$

Gradient Derivation w.r.t. Bias

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \right)^2$$

Differentiate with respect to b:

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_{i=1}^{n} \frac{\partial}{\partial b} \left(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \right)^2$$

Gradient Derivation w.r.t. Bias

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - (\mathbf{w}^T\mathbf{x}_i + b)\right)^2$$

Differentiate with respect to b:

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}\frac{\partial}{\partial b}\left(y_i - (\mathbf{w}^T\mathbf{x}_i + b)\right)^2$$

Apply the chain rule:

$$= \frac{1}{n}\sum_{i=1}^{n} 2(y_i - \hat{y}_i)\frac{\partial}{\partial b}\left(y_i - (\mathbf{w}^T\mathbf{x}_i + b)\right)$$

Since

$$\frac{\partial}{\partial b}\left(y_i - (\mathbf{w}^T\mathbf{x}_i + b)\right) = -1$$

we get

$$\boxed{\frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)}$$

Vector Form of the Gradient

Instead of deriving each weight separately, we can write the gradient compactly.

Vector Form of the Gradient

Instead of deriving each weight separately, we can write the gradient compactly.

Loss:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \right)^2$$

Vector Form of the Gradient

Instead of deriving each weight separately, we can write the gradient compactly.

Loss:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \right)^2$$

Gradient w.r.t. the weight vector:

$$\boxed{\nabla_{\mathbf{w}} \mathcal{L} = -\frac{2}{n} \sum_{i=1}^{n} \mathbf{x}_i (y_i - \hat{y}_i)}$$

Vector Form of the Gradient

Instead of deriving each weight separately, we can write the gradient compactly.

Loss:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - (\mathbf{w}^T \mathbf{x}_i + b)\right)^2$$

Gradient w.r.t. the weight vector:

$$\boxed{\nabla_{\mathbf{w}} \mathcal{L} = -\frac{2}{n} \sum_{i=1}^{n} \mathbf{x}_i (y_i - \hat{y}_i)}$$

Gradient w.r.t. bias:

$$\boxed{\frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)}$$

Vector Form of the Gradient

Instead of deriving each weight separately, we can write the gradient compactly.

Loss:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \left(y_i - (\mathbf{w}^T \mathbf{x}_i + b) \right)^2$$

Gradient w.r.t. the weight vector:

$$\boxed{\nabla_{\mathbf{w}} \mathcal{L} = -\frac{2}{n} \sum_{i=1}^{n} \mathbf{x}_i (y_i - \hat{y}_i)}$$

Gradient w.r.t. bias:

$$\boxed{\frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)}$$

Update rules:

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}} \mathcal{L}$$

$$b \leftarrow b - \eta \frac{\partial \mathcal{L}}{\partial b}$$

Matrix Form (Advanced View)

Let

$$X \in \mathbb{R}^{n \times d}, \qquad \mathbf{y} \in \mathbb{R}^{n}, \qquad \hat{\mathbf{y}} = X\mathbf{w} + b\mathbf{1}$$

Matrix Form (Advanced View)

Let

$$X \in \mathbb{R}^{n \times d}, \qquad \mathbf{y} \in \mathbb{R}^{n}, \qquad \hat{\mathbf{y}} = X\mathbf{w} + b\mathbf{1}$$

Then the loss is

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n}\|\mathbf{y} - \hat{\mathbf{y}}\|_2^2$$

Matrix Form (Advanced View)

Let

$$X \in \mathbb{R}^{n \times d}, \qquad \mathbf{y} \in \mathbb{R}^{n}, \qquad \hat{\mathbf{y}} = X\mathbf{w} + b\mathbf{1}$$

Then the loss is

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n}\|\mathbf{y} - \hat{\mathbf{y}}\|_2^2$$

The gradients become

$$\boxed{\nabla_{\mathbf{w}}\mathcal{L} = -\frac{2}{n}X^T(\mathbf{y} - \hat{\mathbf{y}})}$$

$$\boxed{\frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{n}\mathbf{1}^T(\mathbf{y} - \hat{\mathbf{y}})}$$

Matrix Form (Advanced View)

Let

$$X \in \mathbb{R}^{n \times d}, \qquad \mathbf{y} \in \mathbb{R}^n, \qquad \hat{\mathbf{y}} = X\mathbf{w} + b\mathbf{1}$$

Then the loss is

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{n}\|\mathbf{y} - \hat{\mathbf{y}}\|_2^2$$

The gradients become

$$\boxed{\nabla_{\mathbf{w}}\mathcal{L} = -\frac{2}{n}X^T(\mathbf{y} - \hat{\mathbf{y}})}$$

$$\boxed{\frac{\partial \mathcal{L}}{\partial b} = -\frac{2}{n}\mathbf{1}^T(\mathbf{y} - \hat{\mathbf{y}})}$$

Why this matters:

- This same idea extends directly to neural networks
- Backpropagation is repeated chain rule on larger compositions

Takeaway

- Linear regression models a relationship using a straight line
- Prediction function:

$$\hat{y} = wx + b$$

- The best line minimizes prediction error
- It is simple, interpretable, and foundational for machine learning

Worked Example: Gradient Descent with Multiple Features

Consider the following data points:

X_1	X_2	Y_t
1	2	40
2	3	90

We use the hypothesis:

$$Y_p = w_1 x_1^4 + w_2 x_2^3 + b$$

and the Mean Squared Error loss:

$$\mathcal{L}(w_1, w_2, b) = \frac{1}{n} \sum_{i=1}^{n} (y_{t,i} - y_{p,i})^2$$

Question

Using the data and hypothesis below:

$$Y_p = w_1 x_1^4 + w_2 x_2^3 + b$$

X_1	X_2	Y_t
1	2	40
2	3	90

Answer the following:

1. Calculate the loss magnitude using Mean Squared Error. [3]
2. If $w_1 = 0.3$, $w_2 = 0.4$, and $b = 0.7$, apply one step of gradient descent to update w_2. Use learning rate $\eta = 0.0001$. [5]
3. If Mean Squared Error is replaced by Mean Cubic Error, what potential issues may arise? [2]

Solution Step 1: Compute Predictions

Given:

$$w_1 = 0.3, \quad w_2 = 0.4, \quad b = 0.7$$

Hypothesis:

$$Y_p = w_1 x_1^4 + w_2 x_2^3 + b$$

For data point 1: $(x_1, x_2, y_t) = (1, 2, 40)$

$$y_{p,1} = 0.3(1^4) + 0.4(2^3) + 0.7$$

$$y_{p,1} = 0.3 + 0.4(8) + 0.7 = 0.3 + 3.2 + 0.7 = 4.2$$

Solution Step 1: Compute Predictions

Given:

$$w_1 = 0.3, \quad w_2 = 0.4, \quad b = 0.7$$

Hypothesis:

$$Y_p = w_1 x_1^4 + w_2 x_2^3 + b$$

For data point 1: $(x_1, x_2, y_t) = (1, 2, 40)$

$$y_{p,1} = 0.3(1^4) + 0.4(2^3) + 0.7$$

$$y_{p,1} = 0.3 + 0.4(8) + 0.7 = 0.3 + 3.2 + 0.7 = 4.2$$

For data point 2: $(x_1, x_2, y_t) = (2, 3, 90)$

$$y_{p,2} = 0.3(2^4) + 0.4(3^3) + 0.7$$

$$y_{p,2} = 0.3(16) + 0.4(27) + 0.7 = 4.8 + 10.8 + 0.7 = 16.3$$

Solution Step 2: Compute MSE Loss

Mean Squared Error:

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}(y_{t,i} - y_{p,i})^2$$

Here $n = 2$, so

$$\mathcal{L} = \frac{1}{2}\left[(40 - 4.2)^2 + (90 - 16.3)^2\right]$$

Solution Step 2: Compute MSE Loss

Mean Squared Error:

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}(y_{t,i} - y_{p,i})^2$$

Here $n = 2$, so

$$\mathcal{L} = \frac{1}{2}\left[(40 - 4.2)^2 + (90 - 16.3)^2\right]$$

Compute each term:

$$40 - 4.2 = 35.8, \qquad (35.8)^2 = 1281.64$$

$$90 - 16.3 = 73.7, \qquad (73.7)^2 = 5431.69$$

Solution Step 2: Compute MSE Loss

Mean Squared Error:

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}(y_{t,i} - y_{p,i})^2$$

Here $n = 2$, so

$$\mathcal{L} = \frac{1}{2}\left[(40 - 4.2)^2 + (90 - 16.3)^2\right]$$

Compute each term:

$$40 - 4.2 = 35.8, \qquad (35.8)^2 = 1281.64$$

$$90 - 16.3 = 73.7, \qquad (73.7)^2 = 5431.69$$

Therefore,

$$\mathcal{L} = \frac{1}{2}(1281.64 + 5431.69)$$

$$\mathcal{L} = \frac{6713.33}{2} = 3356.665$$

Solution Step 3: Derive Gradient for w_2

Loss:

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n}(y_{t,i} - y_{p,i})^2$$

where

$$y_{p,i} = w_1 x_{1,i}^4 + w_2 x_{2,i}^3 + b$$

For w_2,

$$\frac{\partial \mathcal{L}}{\partial w_2} = -\frac{2}{n}\sum_{i=1}^{n} x_{2,i}^3 (y_{t,i} - y_{p,i})$$

Since $n = 2$,

$$\frac{\partial \mathcal{L}}{\partial w_2} = -\sum_{i=1}^{2} x_{2,i}^3 (y_{t,i} - y_{p,i})$$

Solution Step 4: Update w_2

We already have:

$$y_{p,1} = 4.2, \qquad y_{p,2} = 16.3$$

Errors:

$$y_{t,1} - y_{p,1} = 40 - 4.2 = 35.8$$

$$y_{t,2} - y_{p,2} = 90 - 16.3 = 73.7$$

Also,

$$x_{2,1}^3 = 2^3 = 8, \qquad x_{2,2}^3 = 3^3 = 27$$

Solution Step 4: Update w_2

We already have:

$$y_{p,1} = 4.2, \qquad y_{p,2} = 16.3$$

Errors:

$$y_{t,1} - y_{p,1} = 40 - 4.2 = 35.8$$

$$y_{t,2} - y_{p,2} = 90 - 16.3 = 73.7$$

Also,

$$x_{2,1}^3 = 2^3 = 8, \qquad x_{2,2}^3 = 3^3 = 27$$

So,

$$\frac{\partial \mathcal{L}}{\partial w_2} = -\left[8(35.8) + 27(73.7)\right]$$

$$= -\left[286.4 + 1989.9\right] = -2276.3$$

Solution Step 4: Update w_2

We already have:

$$y_{p,1} = 4.2, \qquad y_{p,2} = 16.3$$

Errors:

$$y_{t,1} - y_{p,1} = 40 - 4.2 = 35.8$$

$$y_{t,2} - y_{p,2} = 90 - 16.3 = 73.7$$

Also,

$$x_{2,1}^3 = 2^3 = 8, \qquad x_{2,2}^3 = 3^3 = 27$$

So,

$$\frac{\partial \mathcal{L}}{\partial w_2} = -\left[8(35.8) + 27(73.7)\right]$$

$$= -\left[286.4 + 1989.9\right] = -2276.3$$

continue

Gradient descent update:

$$w_2^{\text{new}} = w_2 - \eta \frac{\partial \mathcal{L}}{\partial w_2}$$

$$w_2^{\text{new}} = 0.4 - 0.0001(-2276.3)$$

$$w_2^{\text{new}} = 0.4 + 0.22763 = 0.62763$$

$$\boxed{w_2^{\text{new}} = 0.62763}$$

Solution Step 5: If MSE is Replaced by Mean Cubic Error

Suppose the loss becomes:

$$\mathcal{L}_{\text{cubic}} = \frac{1}{n}\sum_{i=1}^{n}(y_{t,i} - y_{p,i})^3$$

Potential issues:

- **Negative and positive errors may cancel out**, so the loss may not reflect total error properly.
- **Optimization becomes unstable**, because cubic terms can produce very large gradients for large errors.
- **Loss is not always non-negative**, unlike MSE, which makes interpretation harder.

MSE is usually preferred because it is non-negative, smooth, and stable.